



COURSE 2B - DA-TO-DATA-ENGINEER BRIDGE 2026

B01 - ENGINEERING SQL TRANSITION

8-lesson fast-track learner book | Beginner-first professional institute rebuild |

Purpose

Turn SQL you already use as an analyst into SQL that can be trusted as an engineering transformation: explicit grain, inspectable stages, deterministic windows, data-quality rules, reconciliation, cautious plan awareness and maintainable handoff.

Fast-track rule

This module does not reteach basic SELECT / WHERE / GROUP BY / joins from zero. It briefly activates those C1 skills, then teaches the engineering change. If the C1 prerequisite check fails, repair the exact C1 skill rather than stretching B01 into a second beginner SQL course.

Brand: Option B - book/learning -> transition -> pipeline

This book is your teacher

Use one lesson at a time. Do not read the protected review before you save a genuine attempt. Every lesson follows the same beginner-first sequence, even when the engineering idea becomes more technical.

Daily lesson loop

1) Understand why -> 2) picture it -> 3) learn new words -> 4) watch only the assigned segment if useful -> 5) follow once -> 6) close the example -> 7) do a fresh task -> 8) validate -> 9) teach it back -> 10) only after a saved attempt, use protected review -> 11) close review -> 12) changed retry -> 13) save evidence.

Prerequisite check

Before B01, you should independently write SELECT/WHERE/GROUP BY/HAVING, INNER/LEFT joins, simple aggregates, a subquery/CTE, a basic window function, and basic date logic. Course 1 already teaches those. If you cannot do one of them without copying, mark that exact item for C1 repair and return; do not restart all SQL.

Tools and first-use setup

Run **B01_POSTGRESQL_LAB_SETUP_RC1.sql** once in PostgreSQL. This RC1 keeps the proven synthetic lab schema/data from the frozen baseline. PostgreSQL 18 current documentation controls dialect behavior; the preserved Alex video is MySQL-based visual support only.

What B01 deliberately does not teach

Linux/WSL2, Git branches/PR workflow, virtual environments, Python packages, CLI design and Docker are not hidden prerequisites here. The current C3-00 course teaches them explicitly. B01 only builds the SQL reasoning C3 expects.

Contents / Index

This book is your teacher	2
Contents / Index	2
B01 module map	3
B01-L01 - Grain and cardinality contracts for engineering SQL	4
B01-L02 - Staged transformations with CTEs and subqueries	6
B01-L03 - Window patterns for ranking, latest-row and sequence logic	8
B01-L04 - Conditional and date transformations with explicit business rules	10
B01-L05 - Data-quality investigation queries	12
B01-L06 - Reconciliation and validation SQL	14
B01-L07 - EXPLAIN, scans and index awareness	16
B01-L08 - Maintainable deterministic SQL handoff	18
B01 Quick Reference	20

B01 module map

Lesson	Engineering change	C3 handoff
L01	Join -> grain/cardinality contract	C3-02 production SQL + grain enforcement
L02	Complex query -> inspectable stages	C3-02 staging/transformation boundaries
L03	Window analysis -> deterministic sequence logic	C3-02 incremental/latest-row reasoning
L04	KPI/date query -> explicit rule/boundary contract	C3-01 time/incremental boundaries
L05	Dirty data -> repeatable quality diagnostics	C3-02 data tests
L06	Check result -> independent reconciliation	C3-01/C3-02 tests and controls
L07	Slow query guess -> plan evidence	C3-02 performance-aware SQL
L08	Personal query -> maintainable handoff	C3-02 CI/serving handoff

Module exit evidence

All 8 lessons mastered + both Mini-Labs complete + independent Gate A PASS (or explicitly assigned fresh B/C after remediation). A percentage does not substitute for a Gate. B01 remains module until live PostgreSQL runtime is observed on a compatible learner machine.

B01-L01 - Grain and cardinality contracts for engineering SQL

Analyst -> Engineer transition

What you already know: INNER/LEFT JOIN and basic join checks.

What changes: before joining, you declare both grains, expected match pattern and the final grain.

Why engineers care: the query may become a reusable transformation; silent fan-out gets copied everywhere.

Small worked example: aggregate order_items to one row per order before attaching order-level shipping_fee.

Real engineering version: encode row-count, unique-key and reconciliation checks around the transformation.

1. Why it matters

In analysis, a join can look correct because the columns appear and the total seems plausible. In engineering, the same join may feed many downstream jobs. If it silently changes the meaning of one row, every later metric can be wrong. Grain and cardinality are therefore contracts, not vocabulary words.

2. What you are learning

State the input grain on both sides of a join, predict the cardinality, build the intended output grain, and prove that business measures were not multiplied.

3. Prerequisite / what you already know

C1 SA1-SA2: joins, cardinality, row multiplication and join validation.

4. Picture it in your head

Mental model

Imagine two paper lists. Orders has one card per order. Order items has several cards per order. If you staple every item card to its order card, the order card is repeated. Repetition is not automatically wrong; forgetting that the new stack is now item-grain is wrong.

5. New words before we use them

Word	Plain-English meaning
grain	what one row represents
cardinality	how many matching rows can exist on each side of a relationship
fan-out	row multiplication caused by one-to-many or many-to-many matching
invariant	a fact that should remain true, such as a trusted revenue total

6. Watch / visual source

Alex The Analyst, 51:10-1:08:03 (joins) - refresher only. PostgreSQL 18 Table Expressions / Joins is the dialect authority.

[Alex joins visual](#) | [PostgreSQL Table Expressions](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Run three counts before building the final result: orders rows, order_items rows, and distinct order_id in order_items.

Then join orders directly to order_items and observe the repeated order IDs. Finally aggregate items first and return to one row per order.

```
-- Step 1: item grain -> order grain
WITH item_totals AS (
  SELECT order_id,
         SUM(qty * unit_price * (1-discount_pct)) AS item_revenue
  FROM order_items
  GROUP BY order_id
)
SELECT o.order_id, o.booked_region,
       i.item_revenue, o.shipping_fee
FROM orders o
JOIN item_totals i USING (order_id)
ORDER BY o.order_id;
```

8. What you should see / be able to say

You should be able to say: "The direct join is item-grain, so order-level fields repeat. My final query returns one row per order because I aggregate items before the join."

9. Why it worked

The aggregation deliberately changes item grain to order grain before the order-level measure is attached. The output contract is designed before the SELECT list, not inferred afterward.

10. Common beginner mistakes

- Calling every repeated order_id a duplicate. It may be correct at item grain.
- Using DISTINCT to hide fan-out without identifying why rows multiplied.
- Checking only SUM(revenue) while ignoring a multiplied shipping_fee.
- Joining customer history on customer_id alone when multiple history rows can match.

11. If you get stuck

Recovery ladder

Write one sentence for each table: "one row represents ...". Then use COUNT(*), COUNT(DISTINCT key), and a small LIMIT sample. Do not change the join until you can name the exact point where grain changes.

12. Now you do a small version alone

Using a fresh copy of the lab, build one row per completed order with customer name, booked region, item revenue and shipping fee. Also demonstrate one unsafe join to a multi-row table and the control that catches it.

13. Validate your result

Prove final order_id uniqueness; reconcile summed item_revenue to the raw order_items calculation; compare shipping_fee against an order-only control.

14. Teach it back

Explain in 45-60 seconds: input grains -> expected cardinality -> output grain -> one fan-out risk -> one independent control.

15. Protected review + changed retry

Attempt protection

After saving the attempt, open only B01-L01 in the protected review. Close it. Then solve a changed customer-month grain task without using DISTINCT as a repair.

16. Evidence / mastery + next route

Mastery evidence

Save SQL, grain comments, before/after row counts, one reconciliation, one intentionally bad join + detection, and explain-back. Mastery requires a fresh changed task with no unexplained fan-out.

B01-L02 - Staged transformations with CTEs and subqueries

Analyst -> Engineer transition

What you already know: use subqueries/CTEs to solve harder analysis.

What changes: every stage gets a purpose, grain and validation checkpoint.

Why engineers care: staged SQL is easier to debug, test, review and later translate into transformation models.

Small worked example: item rows -> one row/order -> one row/region.

Real engineering version: the same stage boundaries later become dbt staging/intermediate/mart models or pipeline transformations.

1. Why it matters

A long analyst query can be correct today but hard to inspect tomorrow. Engineering SQL must expose where filtering, grain changes and business rules happen so another person can test one stage without mentally executing the whole query.

2. What you are learning

Turn a multi-step requirement into named stages, choose subquery vs CTE deliberately, and validate intermediate outputs before trusting the final result.

3. Prerequisite / what you already know

C1 SA5-SA6: subqueries and CTEs for readable multi-step SQL.

4. Picture it in your head

Mental model

Think of an airport journey: check-in, security, gate, boarding. If a passenger disappears, you want to know at which checkpoint. Named SQL stages give the same traceability to data.

5. New words before we use them

Word	Plain-English meaning
CTE	a named query result defined with WITH and used by the statement
derived table	a subquery placed in FROM and treated like a temporary result
stage	one purposeful transformation step with a stated grain
materialization	whether a database computes/stores an intermediate result in a particular way; not assumed here

6. Watch / visual source

Alex subqueries 1:35:16-1:45:57 and CTEs 1:58:55-2:09:06. PostgreSQL 18 WITH Queries controls current behavior.

[Alex SQL class](#) | [PostgreSQL WITH Queries](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Run base alone and record its grain/count. Run order_level alone and prove one row per order. Only then run the regional summary. Change one filter and predict which stages should change.

```
WITH base AS (
  SELECT o.order_id, o.booked_region,
         i.qty, i.unit_price, i.discount_pct
  FROM orders o
  JOIN order_items i USING (order_id)
  WHERE o.status='Completed'
),
order_level AS (
  SELECT order_id, booked_region,
         SUM(qty*unit_price*(1-discount_pct)) AS revenue
  FROM base
  GROUP BY order_id, booked_region
)
SELECT booked_region, COUNT(*) AS orders, SUM(revenue) AS revenue
FROM order_level
GROUP BY booked_region
ORDER BY booked_region;
```

8. What you should see / be able to say

You should see a clear grain transition: item-like rows in base, one row per completed order in order_level, then one row per region in the final output.

9. Why it worked

Each stage has one job and the grouping columns define the grain change explicitly. Independent stage checks can isolate the first incorrect transformation.

10. Common beginner mistakes

- Using WITH only to make a query longer without naming meaningful stages.
- Assuming a CTE is automatically faster.
- Repeating the same business formula in several stages and letting definitions drift.
- Validating only the final result, so the first bad stage is invisible.

11. If you get stuck

Recovery ladder

Temporarily replace the final SELECT with SELECT * FROM stage_name LIMIT 20. Write the stage grain beside it. Validate that stage before moving forward.

12. Now you do a small version alone

Build staged SQL that finds completed orders above the average completed-order revenue, then produces a regional summary. At least one stage must be independently validated.

13. Validate your result

Check order-level uniqueness, reconcile order-level revenue to raw item revenue, and independently verify one regional total.

14. Teach it back

Explain why a named stage is useful even when it does not make the query faster. Name the grain of every stage.

15. Protected review + changed retry

Attempt protection

After review, solve the same transformation pattern for a changed status/date window and different final grain.

16. Evidence / mastery + next route

Mastery evidence

Save staged SQL, a stage map (name -> purpose -> grain), at least two stage checks, final reconciliation and a changed retry.

B01-L03 - Window patterns for ranking, latest-row and sequence logic

Analyst -> Engineer transition

What you already know: rank rows, calculate region averages, use LAG/LEAD.

What changes: you must defend partition, complete ordering/tie-break, frame and time grain.

Why engineers care: latest-row selection and incremental logic fail when ties or missing periods are ignored.

Small worked example: rank completed orders within region with a stable second ordering key.

Real engineering version: deterministic latest-row logic later supports deduplication, incremental models and change processing.

1. Why it matters

Window functions are powerful precisely because they preserve detail while calculating across related rows. In engineering, that power is risky unless ordering and tie rules are deterministic; the same data must produce the same chosen row every run.

2. What you are learning

Use window calculations for comparison, ranking and sequence logic while stating partition, ordering, frame, tie behavior and missing-period assumptions.

3. Prerequisite / what you already know

C1 SA7-SA8: window functions, PARTITION BY, ranking, latest-row logic and LAG/LEAD.

4. Picture it in your head

Mental model

Imagine runners on separate tracks. PARTITION BY decides which runners share a race; ORDER BY decides their sequence; the frame decides which nearby runners each calculation is allowed to consider.

5. New words before we use them

Word	Plain-English meaning
partition	the group of rows a window function considers together
window order	the sequence used inside OVER(...)
frame	the subset of ordered rows used by a window aggregate
peer/tie	rows equal on the window ordering values
deterministic	same declared inputs produce the same chosen/order result

6. Watch / visual source

Alex window functions 1:45:58-1:58:54. PostgreSQL 18 Window Functions Tutorial is the dialect authority.

[Alex window visual](#) | [PostgreSQL Window Tutorial](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Start with regional average beside each order. Add ranking. Create an intentional tie and compare ordering with and without order_id as a tie-break. Then build monthly revenue and LAG(previous month).

```
WITH order_level AS ( ... )
SELECT order_id, booked_region, revenue,
       AVG(revenue) OVER (PARTITION BY booked_region) AS region_avg,
       ROW_NUMBER() OVER (
         PARTITION BY booked_region
         ORDER BY revenue DESC, order_id
       ) AS stable_rank
FROM order_level
ORDER BY booked_region, stable_rank;
```

8. What you should see / be able to say

Detail rows remain present. Ranking is stable because the ordering is complete. LAG returns the previous observed row, which is not automatically the previous calendar month if a month is missing.

9. Why it worked

Window functions operate after the row set for the query is formed. PARTITION BY groups the calculation, ORDER BY defines sequence, and the tie-break makes row selection reproducible.

10. Common beginner mistakes

- Believing result ORDER BY and window ORDER BY are the same thing.
- Using ROW_NUMBER with an incomplete tie order for latest-row selection.
- Using a running SUM without an explicit frame when peer behavior matters.
- Treating LAG as previous calendar month when missing periods exist.

11. If you get stuck**Recovery ladder**

First remove every window except one. Verify the pre-window row grain. Then write PARTITION, ORDER, FRAME as three separate English decisions before coding.

12. Now you do a small version alone

Create one result that ranks completed customers within region, one running daily completed-revenue query with explicit ROWS frame, and one monthly LAG comparison. State the missing-period rule.

13. Validate your result

Compare row count before/after windows; manually verify one partition/rank; reconcile monthly revenue to completed revenue; create a tie test.

14. Teach it back

Explain why window functions usually preserve rows and why deterministic ordering matters for engineering.

15. Protected review + changed retry**Attempt protection**

After review, perform a changed ranking and sequence task using different grouping and a different tie-break.

16. Evidence / mastery + next route

Mastery evidence

Save SQL, partition/order/frame notes, tie test, missing-period note, manual validation and changed retry.

B01-L04 - Conditional and date transformations with explicit business rules

Analyst -> Engineer transition

What you already know: CASE and date functions for analysis.

What changes: the rule must be written in English first, then encoded with explicit denominator and boundary semantics.

Why engineers care: automated jobs reuse these definitions; ambiguous endpoints create overlap or gaps between runs.

Small worked example: count events in [start, end), then build one row/month with date_trunc.

Real engineering version: the same boundary contract later defines ingestion windows and incremental processing.

1. Why it matters

Business rules often hide inside CASE statements, KPI denominators and date filters. A query can be syntactically correct but answer a different business question because a denominator or endpoint is ambiguous.

2. What you are learning

Translate KPI and time rules from English into explicit SQL, including CASE logic, numerator/denominator filters, monthly grain and half-open timestamp boundaries.

3. Prerequisite / what you already know

C1 SF8 + SA4: CASE classification and date/time analysis.

4. Picture it in your head

Mental model

A ticket checker follows written rules: who is allowed in, which door closes at what time, and which tickets count in the denominator. If the rule is vague, the count is vague.

5. New words before we use them

Word	Plain-English meaning
conditional aggregation	aggregating only rows that satisfy a stated condition
denominator	the population used as the bottom of a rate
half-open interval	start included, end excluded: start <= x < end
date_trunc	PostgreSQL function that rounds a timestamp down to a chosen time grain

6. Watch / visual source

Alex CASE 1:26:39-1:35:15 is optional. PostgreSQL 18 Conditional Expressions and Date/Time Functions are the authorities.

[Alex CASE visual](#) | [PostgreSQL Conditional Expressions](#) | [PostgreSQL Date/Time](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Write the completion-rate numerator and denominator in plain English. Code it by region and validate one region separately. Then run a half-open event window, save count/min/max, and bucket completed orders by month.

```
SELECT booked_region,
       COUNT(*) AS all_orders,
       COUNT(*) FILTER (WHERE status='Completed') AS completed_orders,
       ROUND(
         COUNT(*) FILTER (WHERE status='Completed')::numeric
         / NULLIF(COUNT(*),0), 4
       ) AS completion_rate
FROM orders
GROUP BY booked_region;

SELECT COUNT(*), MIN(event_ts), MAX(event_ts)
FROM order_events
WHERE event_ts >= TIMESTAMP '2026-01-15 00:00:00'
AND event_ts <  TIMESTAMP '2026-02-01 00:00:00';
```

8. What you should see / be able to say

Your rate has an explicit population. Events exactly at the start are included; events exactly at the end are excluded. Monthly output has one row per observed month.

9. Why it worked

The SQL mirrors a written contract. FILTER/CASE makes category conditions visible, NULLIF protects a zero denominator, and the half-open range prevents double counting when adjacent windows meet.

10. Common beginner mistakes

- Mixing all orders and completed orders in an undocumented denominator.
- Using <= on both adjacent window endpoints and double-counting boundary events.
- Converting timestamps to strings for filtering.
- Assuming a missing month means zero rather than absent data.

11. If you get stuck

Recovery ladder

Write examples for one row that should be included and one that should be excluded. Test them before the full query. For KPIs, calculate one region manually.

12. Now you do a small version alone

Produce regional status counts/rate and a separate event-window + monthly revenue result for changed dates. State business rules before SQL.

13. Validate your result

Independent single-region control; event min/max must fit the declared interval; monthly sum must reconcile to the same filtered completed total.

14. Teach it back

Explain the numerator, denominator, start boundary, end boundary and monthly grain without reading SQL.

15. Protected review + changed retry

Attempt protection

After review, solve a changed KPI/date window with a different status rule and different dates.

16. Evidence / mastery + next route

Mastery evidence

Save written rules, SQL, manual denominator check, boundary evidence, monthly reconciliation and changed retry.

B01-L05 - Data-quality investigation queries

Analyst -> Engineer transition

What you already know: find duplicates, NULLs and bad values.

What changes: define the violated business rule, quantify impact and choose severity/owner/action.

Why engineers care: quality checks become automated gates in pipelines and transformation tests.

Small worked example: group order_events by business event key and flag COUNT(*) > 1.

Real engineering version: the check later becomes a blocking test, quarantine route, monitoring metric or incident signal.

1. Why it matters

An analyst may notice a strange row and fix the local output. An engineer needs a repeatable query that exposes the violated rule, counts impact, records severity and leaves an evidence trail for repair.

2. What you are learning

Write diagnostics for duplicate business events, orphan keys, invalid numeric ranges and temporal/history conflicts; classify each issue as BLOCK or WARN with rationale.

3. Prerequisite / what you already know

C1 taught validation and cleaning; this lesson turns checks into repeatable engineering diagnostics.

4. Picture it in your head

Mental model

A smoke alarm does not repaint the wall. It detects a condition, raises evidence and tells you which room needs attention. Quality SQL should expose problems before anyone hides them.

5. New words before we use them

Word	Plain-English meaning
business key	columns that identify an event/entity according to business meaning
orphan	child row with no matching parent
quality rule	a testable statement describing valid data
BLOCK	defect severe enough to stop trusted publication
WARN	defect recorded for review but allowed under a stated policy

6. Watch / visual source

Alex data-cleaning project 2:42:46-3:32:13 is optional workflow context only. The Bridge lab is the authority for quality rules.

[Alex cleaning project visual](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Run four diagnostic families: duplicate business event, orphan event, invalid qty/price/discount, and customer history with more than one open/current row. For each, write the rule before the query and the consequence after it.

```
-- Duplicate business event rule
SELECT order_id, event_type, event_ts, COUNT(*) AS n
FROM order_events
GROUP BY order_id, event_type, event_ts
HAVING COUNT(*) > 1;

-- Orphan rule
SELECT e.*
FROM order_events e
LEFT JOIN orders o USING (order_id)
WHERE o.order_id IS NULL;
```

8. What you should see / be able to say

Each diagnostic returns only violating rows/groups, not a "cleaned" output. You can state how many defects exist and why each matters.

9. Why it worked

The query is designed from the rule, not from a desired clean result. That makes the check reusable and auditable.

10. Common beginner mistakes

- SELECT DISTINCT to make symptoms disappear.
- Deleting an orphan before confirming whether the parent is late, missing or invalid.
- Treating numeric type validity as business validity.
- Calling everything BLOCK without considering downstream impact and policy.

11. If you get stuck

Recovery ladder

For one suspicious row, write: expected rule -> observed evidence -> downstream risk -> proposed severity -> owner. Then write SQL that finds every row breaking the same rule.

12. Now you do a small version alone

Create a quality report with four rules, violating-row counts, sample evidence, BLOCK/WARN classification and one next action per rule.

13. Validate your result

Manually inspect at least one returned defect; prove a known good row is not flagged; run a second count/control for each rule family.

14. Teach it back

Explain why a quality query should expose the rule violation rather than hide it with DISTINCT or COALESCE.

15. Protected review + changed retry

Attempt protection

After review, define a fresh duplicate/orphan/range/history rule on changed keys or thresholds.

16. Evidence / mastery + next route

Mastery evidence

Save rule table, SQL, defect counts, severity rationale, sample evidence, owner/action and changed retry.

B01-L06 - Reconciliation and validation SQL

Analyst -> Engineer transition

What you already know: check row counts and totals.

What changes: choose controls that can catch the specific failure mode and locate the first broken stage.

Why engineers care: source-to-output reconciliation is how scheduled jobs prove completeness/correctness.

Small worked example: compare order-level item revenue with a direct raw item total.

Real engineering version: the same control becomes a post-load test, run metric, release gate or replay validation.

1. Why it matters

A successful query is not proof. The strongest validation follows a different path from the transformation, so it has a chance to disagree when the transformation is wrong.

2. What you are learning

Design independent row, key and financial controls; debug the earliest failed contract; distinguish reconciliation from repeating the same logic.

3. Prerequisite / what you already know

C1 SA2 + PBM6: join validation and model trust checks.

4. Picture it in your head

Mental model

Two cashiers count the same till independently. If both copy the same mistaken receipt total, agreement proves nothing. Independent methods are valuable because they can disagree.

5. New words before we use them

Word	Plain-English meaning
reconciliation	comparing independently derived controls that should agree
control total	trusted count/sum used to test an output
independent check	validation that does not simply repeat the same risky logic
first failing stage	earliest transformation where an invariant stops holding

6. Watch / visual source

Course-built lesson and lab. No external video is required; this is a reasoning skill best learned by running wrong-but-plausible queries.

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Take three broken baseline queries: syntax, aggregate filter in WHERE, and history fan-out. Repair one at a time. For each, write a control that would have caught the original defect even if the result looked plausible.

```
-- Transformation output control
SELECT SUM(item_revenue) FROM order_level;

-- Independent raw control
SELECT SUM(qty * unit_price * (1-discount_pct))
FROM order_items i
JOIN orders o USING (order_id)
WHERE o.status='Completed';
```

8. What you should see / be able to say

You should be able to point to the first failed contract and show a control that changes from PASS to FAIL when the bad logic is restored.

9. Why it worked

Validation is tied to a failure hypothesis. It does not merely compare the result to itself.

10. Common beginner mistakes

- Running the same query twice and calling it validation.
- Only checking that the row count is non-zero.
- Fixing a downstream summary instead of the earlier grain defect.
- Ignoring unmatched keys because the total still happens to reconcile.

11. If you get stuck

Recovery ladder

Name the suspected failure: missing rows, extra rows, multiplied amount, wrong category, wrong time window. Pick a control designed to detect that exact failure.

12. Now you do a small version alone

Repair the three broken queries in the practice file and add an independent control for each. Add one source-to-output completed-revenue reconciliation.

13. Validate your result

Temporarily restore the bad logic and prove the chosen control fails. Then restore the repair and prove it passes.

14. Teach it back

Explain the difference between "my query ran" and "my output is reconciled" with one concrete example.

15. Protected review + changed retry

Attempt protection

After review, diagnose a fresh plausible-but-wrong summary with a different fan-out/filter defect.

16. Evidence / mastery + next route

Mastery evidence

Save bad query, symptom, first failing stage, repaired query, independent control before/after and changed retry.

B01-L07 - EXPLAIN, scans and index awareness

Analyst -> Engineer transition

What you already know: a query can be slow.

What changes: inspect the plan before proposing a fix and keep correctness ahead of performance.

Why engineers care: pipelines and transformations can become expensive at scale; plan awareness prevents superstition.

Small worked example: EXPLAIN a selective customer/status/date query before and after a candidate composite index.

Real engineering version: C3 later deepens performance-aware SQL and query-plan reasoning.

1. Why it matters

Performance problems appear later when data grows. The Bridge goal is not tuning mastery; it is learning to look at PostgreSQL plan evidence without guessing or adding indexes blindly.

2. What you are learning

Run EXPLAIN, recognize common scan/join plan vocabulary, compare estimates, propose a candidate index from the query shape, and avoid unmeasured performance claims.

3. Prerequisite / what you already know

C1 had only extended SQL awareness. Reading PostgreSQL plans is genuinely new Bridge content.

4. Picture it in your head

Mental model

A route planner can choose highway or local roads based on distance and traffic. PostgreSQL chooses a plan based on query structure and statistics. A different route is not automatically worse; you inspect evidence.

5. New words before we use them

Word	Plain-English meaning
planner	PostgreSQL component that chooses an execution plan
Seq Scan	reading table pages sequentially
Index Scan	using an index access path to find matching rows
cost	planner estimate used to compare plans; not wall-clock milliseconds
estimated rows	planner prediction of how many rows a plan node will produce

6. Watch / visual source

Primary: PostgreSQL 18 Using EXPLAIN. Optional visual: Dr. Ibrar Ahmed, 00:00-07:00 only, for cost and estimated-vs-actual rows.

[PostgreSQL Using EXPLAIN](#) | [Optional EXPLAIN visual](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Run EXPLAIN on the supplied query. Record scan type and estimated rows. Create the candidate index, run EXPLAIN again, and record what changed. Do not use EXPLAIN ANALYZE unless you understand that it executes the statement.


```
EXPLAIN
SELECT order_id, customer_id, order_date
FROM orders
WHERE status='Completed'
      AND customer_id = 4
      AND order_date >= DATE '2026-01-01';

-- Candidate only after query-shape reasoning
CREATE INDEX idx_orders_customer_status_date
ON orders (customer_id, status, order_date);
```

8. What you should see / be able to say

On a small training table PostgreSQL may still prefer a sequential scan. That is acceptable. Your evidence is the plan comparison and reasoning, not a forced index scan.

9. Why it worked

The planner chooses among access paths based on estimated cost. Index usefulness depends on selectivity, table size, ordering and workload, not on the mere existence of a WHERE clause.

10. Common beginner mistakes

- Calling Seq Scan "bad" by default.
- Saying an index made the query faster without measuring runtime.
- Creating many indexes without considering write/storage cost.
- Reading cost as milliseconds.

11. If you get stuck

Recovery ladder

Ignore most plan details. First locate the scan node, filter/index condition and estimated rows. Then ask whether the table is small and the predicate selective.

12. Now you do a small version alone

EXPLAIN a changed selective query, propose one candidate index, compare the second plan, and write a 4-line evidence note.

13. Validate your result

The query result is unchanged by the index; plan observations are factual; any speed statement is either measured or explicitly not claimed.

14. Teach it back

Explain cost, Seq Scan, Index Scan and estimated rows in simple language. State why a Seq Scan can be reasonable.

15. Protected review + changed retry

Attempt protection

After review, inspect a different filter/order query and decide whether an index proposal is justified.

16. Evidence / mastery + next route

Mastery evidence

Save query, before/after plans, index rationale, unchanged correctness control, no-unmeasured-claim statement and changed retry.

B01-L08 - Maintainable deterministic SQL handoff

Analyst -> Engineer transition

What you already know: write a validated analysis and explain findings.

What changes: document contracts and operational failure risks so the SQL is reusable by someone else.

Why engineers care: maintainability and reproducibility are part of correctness.

Small worked example: raw -> order_level -> monthly_region with grain comments and separate reconciliation.

Real engineering version: C3-02 turns these habits into production-style SQL, dbt models, tests, CI and curated serving handoff.

1. Why it matters

The final Bridge step changes ownership: the SQL is no longer only for your answer. Another engineer should understand what it expects, rerun it, detect failure and modify it without guessing your hidden assumptions.

2. What you are learning

Package SQL as a small transformation handoff with purpose, input/output grain, named stages, deterministic ordering, assumptions, quality controls, reconciliation, limitation and owner/next step.

3. Prerequisite / what you already know

C1 M-SQLA already required validated multi-table analysis and evidence.

4. Picture it in your head

Mental model

Think of leaving a machine for the next shift. A working machine is not enough; the next operator needs labels, checks, known risks and instructions for what to do when a warning light appears.

5. New words before we use them

Word	Plain-English meaning
deterministic order	an ORDER BY that fully specifies row order when order matters
handoff	evidence/instructions allowing another person to operate or modify work safely
assumption	condition your logic relies on but may not enforce itself
owner	person/team responsible for resolving a rule, data or operational issue

6. Watch / visual source

Optional Alex project workflow 2:42:46-3:32:13. PostgreSQL 18 ORDER BY is the authority for deterministic result ordering.

[PostgreSQL ORDER BY](#) | [Alex project workflow visual](#)

Do this after watching

Do not continue because the video ended. Return to the local lab and perform the exact follow-along below from your own keyboard. Save evidence from the course data, not a screenshot of the video.

7. Follow with me once

Refactor one working multi-step query into named stages. Add a header contract, explicit grain comments, one deterministic ORDER BY, one separate reconciliation, one known limitation and an owner/next-step note.

```
-- PURPOSE: trusted completed revenue by region-month
-- INPUT: order_items = one row per order item
-- OUTPUT: one row per booked_region + month
WITH order_level AS ( ... ),
monthly_region AS ( ... )
SELECT booked_region, month, orders, revenue
FROM monthly_region
ORDER BY month, booked_region;

-- Separate reconciliation query follows below.
```

8. What you should see / be able to say

A reviewer can identify input/output meaning before reading every expression, run the validation separately, and know what remains uncertain.

9. Why it worked

The handoff separates transformation logic from its contract and evidence. Deterministic ordering avoids accidental dependence on storage/plan order when output order matters.

10. Common beginner mistakes

- Using SELECT * in a durable handoff without an explicit schema reason.
- Relying on unstated default row order.
- Writing comments that repeat syntax instead of explaining business meaning.
- Leaving no clue who should resolve an unmatched key or changed business rule.

11. If you get stuck

Recovery ladder

Use this order: PURPOSE -> INPUT GRAIN -> OUTPUT GRAIN -> ASSUMPTIONS -> STAGES -> VALIDATION -> LIMITATION -> OWNER/NEXT STEP. Fill each line before polishing SQL.

12. Now you do a small version alone

Produce the regional-month completed-revenue handoff in the practice file, then give it a changed customer-segment/month version without reopening the review.

13. Validate your result

Reconciliation agrees; order is deterministic; all stage grains are stated; a new reader can identify failure risks and next action.

14. Teach it back

Give a 60-second handoff as if speaking to the engineer who will own the query tomorrow.

15. Protected review + changed retry

Attempt protection

After review, create a changed handoff with a different final grain and at least one different quality risk.

16. Evidence / mastery + next route

Mastery evidence

Save final SQL, header contract, stage map, reconciliation, quality note, plan/limitations where relevant, owner/next step and changed retry.

B01 Quick Reference

Control	Ask before you trust the result
Grain	What does one input row mean? What must one output row mean?
Cardinality	Can this relationship multiply rows or measures?
Stage	Where does filtering, business logic or grain change?
Window	Partition, order, frame, ties and missing periods are explicit?
Time	Are start/end/timezone/grain rules explicit?
Quality	Which rule is violated? BLOCK or WARN?
Validation	Could this independent check catch the original defect?
EXPLAIN	What plan evidence is visible? Am I avoiding an unmeasured speed claim?
Handoff	Can another person rerun, validate, modify and recover this safely?

After B01

Complete Mini-Lab 1 (Revenue Reliability Pack), Mini-Lab 2 (Quality + Plan Review), then attempt Gate A independently. If Gate A passes, B01 ends. Gate B/C remain sealed unless a failed attempt plus remediation requires an explicit fresh full reassessment.